

Индивидуальный проект. Этап 3

**Описание и программная реализация проекта.: Образование
планетной системы.**

Тойчубекова А.Н Четвергова М.В Просина К.М.
Чигладзе М.В. Митрофанов Т.А.

Содержание

1	Назначение работы	5
2	Состав программного комплекса	7
3	Описание программной реализации	8
3.1	Язык программирования и зависимости	8
3.2	Структуры данных	9
3.3	Основные функции и модули	9
4	Параметры моделирования	20
5	Результаты расчёта	22
5.1	Эволюция протопланетного диска	22
5.2	Сохраняющиеся величины	27
6	Интерпретация результатов	32
7	Заключение	33

Список иллюстраций

5.1	Начальное состояние диска ($t = 0, N = 250$).	22
5.2	Состояние диска ($t = 8, N = 59$).	23
5.3	Состояние диска ($t = 16, N = 32$).	24
5.4	Состояние диска ($t = 24, N = 15$).	25
5.5	Состояние диска ($t = 32, N = 12$).	26
5.6	Финальное состояние ($t = 40, N = 11$).	27
5.7	Сводный график: $E(t), L_z(t)$ и $N(t)$	28
5.8	Полная механическая энергия $E(t)$	29
5.9	Момент импульса $L_z(t)$	30
5.10	Число активных частиц $N(t)$	31

Список таблиц

3.1	Соответствие алгоритмических блоков и функций кода	10
4.1	Параметры базового расчёта	20

1 Назначение работы

Цель третьего этапа — программная реализация алгоритма, разработанного на втором этапе, в виде работающего расчётного кода и получение визуальных результатов моделирования эволюции протопланетного диска.

В качестве языка реализации выбран Julia 1.10 — это современный высокопроизводительный язык, специально ориентированный на научные вычисления. Используется проектная инфраструктура DrWatson.jl: она автоматически создаёт стандартизованные каталоги (`data/`, `plots/`, `scripts/`), что облегчает воспроизводимость расчётов и хранение результатов.

Что реализовано:

- Инициализация N частиц равномерно по диску радиуса R_0 с малой вертикальной толщиной.
- Кеплеровские начальные скорости — каждая частица движется по круговой орбите вокруг центральной массы.
- Гравитационное взаимодействие $O(N^2)$ между всеми парами частиц со сглаживанием (softening) для устойчивости.
- Упругое отталкивание при контакте по формуле $F = k \cdot \left(\left(\frac{a}{b}\right)^8 - 1\right)$.
- Диссипативное трение при контакте, отбирающее часть тангенциальной энергии.
- Скоростной алгоритм Верле для интегрирования уравнений движения.
- Аккреция (слияние) частиц при контакте и низкой относительной скорости.
- Контроль сохраняющихся величин — полной энергии и момента импульса.

- Сохранение PNG-снимков состояния системы в моменты времени и графиков сохраняющихся величин.

2 Состав программного комплекса

Программный комплекс поставляется в виде единственного файла `protoplanetary_disk.jl` ($\approx 14\,800$ байт, 300+ строк кода на Julia). Файл полностью самодостаточен: все параметры, структуры данных, алгоритмы и процедуры вывода результатов сосредоточены в одном месте, что облегчает понимание, проверку и воспроизведение расчёта.

Структура проекта (DrWatson):

- `Project.toml`, `Manifest.toml` — описание окружения Julia;
- `scripts/protoplanetary_disk.jl` — основной расчётный код;
- `plots/protoplanetary_disk/` — каталог PNG-результатов (создаётся автоматически);
- `data/` — каталог для числовых данных (резерв для расширений).

3 Описание программной реализации

3.1 Язык программирования и зависимости

Язык Julia 1.10 выбран по следующим соображениям: производительность, сопоставимая с C/Fortran, при высокоуровневом синтаксисе; встроенная поддержка линейной алгебры и числовых методов; богатая экосистема пакетов для научных вычислений.

Внешние зависимости:

- `DrWatson.jl` — управление проектом, стандартизованные пути;
- `Plots.jl` — построение и сохранение графиков в PNG;
- `Random.jl`, `LinearAlgebra.jl` — стандартная библиотека Julia.

Подключение пакетов и активация проекта выполняются в начале файла:

```
using DrWatson
@quickactivate "project"

using Random
using Printf
using Plots
using LinearAlgebra
```

3.2 Структуры данных

Центральным объектом модели является изменяемая структура `Particle`, содержащая координаты (x, y) , компоненты скорости (v_x, v_y) , компоненты ускорения (a_x, a_y) , массу m , радиус r и флаг активности `alive`. Все частицы хранятся в векторе `Vector{Particle}`; деактивированные (поглощённые) частицы остаются в массиве с флагом `alive = false`, что исключает динамическое перераспределение памяти в ходе счёта.

```
mutable struct Particle
    x::Float64; y::Float64
    vx::Float64; vy::Float64
    ax::Float64; ay::Float64
    m::Float64          # 8x8x8x8
    r::Float64          # 8x8x8x8
    alive::Bool         # 8x8x8x8 (8x 8x8x8x8x8)
end
```

Радиус частицы вычисляется из массы и плотности по формуле

$$r = \left(\frac{3m}{4\pi\rho} \right)^{1/3},$$

то есть частицы предполагаются сферическими с постоянной объёмной плотностью. В коде это однострочная вспомогательная функция:

```
radius_from_mass(m, ρ) = (3m / (4ρ * π))^(1/3)
```

3.3 Основные функции и модули

Таблица Таблица 3.1 содержит соответствие между алгоритмическими блоками, описанными на Этапе 2, и функциями программного кода.

Таблица 3.1: Соответствие алгоритмических блоков и функций кода

Компонент алгоритма	Реализация в коде
Инициализация диска	<code>init_disk()</code> : равномерное распределение по площади, кеплеровские скорости
Расчёт ускорений	<code>compute_accelerations!()</code> : гравитация звезды + попарная + отталкивание + трение
Интегрирование	<code>verlet_step!()</code> : скоростной алгоритм Верле
Акреция	<code>process_mergers!()</code> : слияние при контакте и $v_{\text{rel}} < V_{\text{MERGE}}$
Диагностика	<code>total_energy()</code> , <code>total_angular_momentum()</code>
Визуализация	<code>plot_snapshot()</code> , <code>savefig()</code>

3.3.1 Инициализация: `init_disk()`

Функция `init_disk()` принимает генератор случайных чисел `MersenneTwister(SEED)` и создаёт N_0 частиц, равномерно распределённых по площади диска радиуса R_0 . Равномерность по площади обеспечивается взятием квадратного корня из случайного числа $\xi_1 \in [0, 1]$: $r = R_0 \sqrt{\xi_1}$. Угол берётся равномерно: $\varphi = 2\pi\xi_2$.

Начальные скорости соответствуют кеплеровским круговым орбитам: $v_{\text{circ}} = \sqrt{GM_\star/r}$. Направление скорости — перпендикулярно радиус-вектору, что обеспечивает вращение диска против часовой стрелки.

```
function init_disk(rng::AbstractRNG)
    parts = Particle[]
```

```

m_each = M_TOTAL / N0
r_each = radius_from_mass(m_each, DENSITY)
for _ in 1:N0
    r1 = rand(rng); r2 = rand(rng)
    # [commented out]
    rr = R0 * sqrt(r1)
    phi = 2pi * r2
    x = rr * cos(phi)
    y = rr * sin(phi)
    # [commented out]
    v_circ = sqrt(G * M_STAR / max(rr, 1e-3))
    vx = -v_circ * sin(phi)
    vy = v_circ * cos(phi)
    push!(parts, Particle(x, y, vx, vy, 0.0, 0.0, m_each, r_each))
end
return parts
end

```

3.3.2 Расчёт ускорений: `compute_accelerations!()`

Функция `compute_accelerations!()` реализует три вида сил:

- **Гравитация центральной звезды** (расположена в начале координат): $\mathbf{a} = -GM_\star \hat{r}/r^2$, со сглаживанием $r^2 \rightarrow r^2 + \varepsilon^2$, где $\varepsilon = \text{EPS_SOFT}$;
- **Попарная гравитация** между частицами $O(N^2)$: аналогично, с тем же ε -сглаживанием;
- **Упругое отталкивание и трение** при контакте $r < r_i + r_j$: $F_{\text{rep}} = K_{\text{REP}} \cdot \left(\left(\frac{a_{\text{sum}}}{r} \right)^8 - 1 \right) \hat{r}$; трение: $F_{\text{fric}} = \beta_{\text{fric}} \cdot F_{\text{rep}} \cdot v_{\text{tg}}$.

Двойной цикл по парам $i < j$ отвечает третьему закону Ньютона: каждая

пара обрабатывается один раз, ускорения добавляются обеим частицам с противоположными знаками.

```
function compute_accelerations!(parts::Vector{Particle})
    n = length(parts)
    for i in 1:n                                     # [XXXXXXXXXX] [XXXXXXXXXX]
        parts[i].ax = 0.0; parts[i].ay = 0.0
    end

    for i in 1:n                                     # [XXXXXXXXXX] [XXXXXXXXXXXXXX] [XXXXXX]
        pi = parts[i]
        pi.alive || continue
        rsq = pi.x^2 + pi.y^2 + EPS_SOFT^2
        r3 = rsq * sqrt(rsq)
        pi.ax += -G * M_STAR * pi.x / r3
        pi.ay += -G * M_STAR * pi.y / r3
    end

    for i in 1:n-1                                   # [XXXXXXXXXX] [XXXXXXXXXXXXXXXXXXXX] O(N^2)
        pi = parts[i]
        pi.alive || continue
        for j in i+1:n
            pj = parts[j]
            pj.alive || continue
            dx = pj.x - pi.x; dy = pj.y - pi.y
            r2 = dx*dx + dy*dy + EPS_SOFT^2; r = sqrt(r2)

            # [XXXXXXXXXXXXXX]
            invr3 = 1.0 / (r2 * r)
        end
    end
end
```

```

fg_x  = G * pi.m * pj.m * dx * invr3
fg_y  = G * pi.m * pj.m * dy * invr3
pi.ax += fg_x / pi.m;  pj.ax -= fg_x / pj.m
pi.ay += fg_y / pi.m;  pj.ay -= fg_y / pj.m

# 计算两质点间的距离 r
if r < pi.r + pj.r
    ratio = (pi.r + pj.r) / max(r, 1e-6)
    fmag  = K_REP * (ratio^8 - 1.0)
    fx = fmag * dx / r;  fy = fmag * dy / r
    pi.ax -= fx / pi.m;  pj.ax += fx / pj.m
    pi.ay -= fy / pi.m;  pj.ay += fy / pj.m

    # 计算两质点间的角动量 L
    wx = pi.vx - pj.vx;  wy = pi.vy - pj.vy
    nx = dx / r;  ny = dy / r
    wn  = wx*nx + wy*ny
    wtx = wx - wn*nx;  wty = wy - wn*ny
    pi.ax -= BETA_FRIC * fmag * wtx / pi.m
    pi.ay -= BETA_FRIC * fmag * wty / pi.m
    pj.ax += BETA_FRIC * fmag * wtx / pj.m
    pj.ay += BETA_FRIC * fmag * wty / pj.m
end
end
end
end

```

3.3.3 Интегрирование: `verlet_step!()`

Реализован скоростной алгоритм Верле (Velocity Verlet), обеспечивающий второй порядок точности по времени при симплектической структуре:

$$x(t + \Delta t) = x(t) + v(t) \Delta t + \frac{1}{2} a(t) \Delta t^2,$$

$$a(t + \Delta t) = f(x(t + \Delta t))/m,$$

$$v(t + \Delta t) = v(t) + \frac{1}{2}(a(t) + a(t + \Delta t))\Delta t.$$

```
function verlet_step!(parts::Vector{Particle}, dt::Float64)
    # 1: Обновить позиции
    for p in parts
        p.alive || continue
        p.x += p.vx * dt + 0.5 * p.ax * dt^2
        p.y += p.vy * dt + 0.5 * p.ay * dt^2
    end

    # 2: Сохранить старые ускорения
    ax_old = [p.ax for p in parts]
    ay_old = [p.ay for p in parts]

    # 3: Вычислить новые ускорения
    compute_accelerations!(parts)

    # 4: Обновить скорости
    for (i, p) in enumerate(parts)
        p.alive || continue
        p.vx += 0.5 * (ax_old[i] + p.ax) * dt
        p.vy += 0.5 * (ay_old[i] + p.ay) * dt
    end
end
```

3.3.4 Аккреция: `process_mergers!()`

После каждого шага интегрирования вызывается `process_mergers!()`. Для каждой пары активных частиц (i, j) проверяются два условия: контакт $r < r_i + r_j$ и малая относительная скорость $|\Delta v| < V_{\text{MERGE}}$. При выполнении обоих условий i поглощает j : координаты и скорости пересчитываются по закону сохранения импульса центра масс, масса суммируется, радиус нового тела берётся из условия сохранения объёма $r^3 = r_i^3 + r_j^3$, а частица j деактивируется (`alive = false`).

```
function process_mergers!(parts::Vector{Particle})
    n = length(parts)
    merged_into = zeros{Int, n}
    for i in 1:n-1
        parts[i].alive || continue
        merged_into[i] == 0 || continue
        for j in i+1:n
            parts[j].alive || continue
            merged_into[j] == 0 || continue
            dx = parts[j].x - parts[i].x
            dy = parts[j].y - parts[i].y
            r = sqrt(dx*dx + dy*dy)
            if r < parts[i].r + parts[j].r
                wmag = sqrt((parts[i].vx - parts[j].vx)^2 +
                           (parts[i].vy - parts[j].vy)^2)
                if wmag < V_MERGE
                    # XXXXXXXXXX: i XXXXXXXXXXXX j XX XXXXXXX XXXXXXXXXXXXXX XXXXXXXXXXXX
                    mi = parts[i].m; mj = parts[j].m; M = mi + mj
                    parts[i].x = (mi*parts[i].x + mj*parts[j].x)
                    parts[i].y = (mi*parts[i].y + mj*parts[j].y)
                    parts[i].vx = (mi*parts[i].vx + mj*parts[j].vx)
```

```

        parts[i].vy = (mi*parts[i].vy + mj*parts[j].vy)
        parts[i].m = M
        # XXXXXXXXXXXX XXXXXXX:  $R^3 = R_{\text{new}}^3 + R_{\text{old}}^3$ 
        parts[i].r = (parts[i].r^3 + parts[j].r^3)^(1/3)
        parts[j].alive = false
        merged_into[j] = i
    end
end
end
end
end

```

3.3.5 Диагностические функции

Функции `total_energy()` и `total_angular_momentum()` вычисляют соответственно полную механическую энергию системы (кинетическая + потенциальная относительно звезды + попарная потенциальная) и полный момент импульса

$$L_z = \sum_i m_i (x_i v_{yi} - y_i v_{xi}).$$

Обе величины записываются каждые 10 шагов и служат критерием корректности численной схемы.

```

function total_energy(parts::Vector{Particle})
    E_k = 0.0; E_p = 0.0
    for p in parts
        p.alive || continue
        E_k += 0.5 * p.m * (p.vx^2 + p.vy^2)
        r_star = sqrt(p.x^2 + p.y^2 + EPS_SOFT^2)
        E_p += -G * M_STAR * p.m / r_star
    end
end

```

```

end
for i in 1:length(parts)-1          # 1000000 1000000000000 1000000
    parts[i].alive || continue
    for j in i+1:length(parts)
        parts[j].alive || continue
        dx = parts[j].x - parts[i].x
        dy = parts[j].y - parts[i].y
        rr = sqrt(dx*dx + dy*dy + EPS_SOFT^2)
        E_p += -G * parts[i].m * parts[j].m / rr
    end
end
return E_k + E_p
end

function total_angular_momentum(parts::Vector{Particle})
    L = 0.0
    for p in parts
        p.alive || continue
        L += p.m * (p.x * p.vy - p.y * p.vx)
    end
    return L
end
end

```

3.3.6 Визуализация: `plot_snapshot()`

Функция строит scatter-диаграмму активных частиц, масштабируя размер маркера пропорционально массе тела, и добавляет жёлтый маркер центральной звезды. Готовый кадр сохраняется в PNG через `savefig`.

```

function plot_snapshot(parts::Vector{Particle}, t::Float64, fname::String)
    xs = [p.x for p in parts if p.alive]
    ys = [p.y for p in parts if p.alive]
    ms = [p.m for p in parts if p.alive]
    sizes = 4 .+ 60 .* (ms ./ maximum(ms))    #  $\text{size} \propto m$ 
    plt = scatter(xs, ys;
        markersize=sizes, markeralpha=0.7, color=:dodgerblue,
        markerstrokewidth=0, legend=false,
        xlabel="x", ylabel="y",
        title=@sprintf("Snapshot at t = %.2f, N = %d", t, length(parts)),
        xlim=(-1.6, 1.6), ylim=(-1.6, 1.6),
        size=(700, 700), aspect_ratio=:equal)
    scatter!(plt, [0.0], [0.0];                # center of mass
        markersize=14, color=:gold,
        markerstrokecolor=:orange, markerstrokewidth=2, label="")
    savefig(plt, fname)
end

```

3.3.7 Главный цикл: `main()`

Функция `main()` организует весь расчёт: инициализирует диск, запускает цикл по шагам времени, на каждом шаге вызывает `verlet_step!()` и `process_mergers!()`, каждые 10 шагов записывает диагностику, а в заданные моменты сохраняет снимки. По завершении строит и сохраняет графики сохраняющихся величин.

```

function main()
    rng    = MersenneTwister(SEED)
    parts = init_disk(rng)

```

```

compute_accelerations!(parts)          # ██████████ ██████████

n_steps    = Int(round(T_MAX / DT))
snap_idxxs = Int.(round.(range(0, n_steps; length=SNAPSHOTS)))

times = Float64[]; energies = Float64[]
angmom = Float64[]; counts = Int[]

for step in 0:n_steps
    t = step * DT
    if step in snap_idxxs          # ██████████ ██████████
        plot_snapshot(parts, t, ...)
    end
    if step % 10 == 0              # ████████ ████████████████
        push!(times, t)
        push!(energies, total_energy(parts))
        push!(angmom,    total_angular_momentum(parts))
        push!(counts,    count(p -> p.alive, parts))
    end
    verlet_step!(parts, DT)        # ████ ████████████████████
    process_mergers!(parts)        # ██████████ ██████████
end
# ... ████████████████ × ████████████████ ██████████ ██████████
end

```

4 Параметры моделирования

Все ключевые параметры вынесены в начало файла как именованные константы, что делает их изменение очевидным и безопасным. Таблица Таблица 4.1 содержит полный перечень параметров и их значения, использованные в базовом расчёте.

Таблица 4.1: Параметры базового расчёта

Параметр	Значение	Описание
N0	250	начальное число частиц
M_TOTAL	0.04	суммарная масса диска (в долях M_*)
DENSITY	30.0	плотность вещества (определяет радиус частиц)
EPS_SOFT	0.05	параметр сглаживания гравитации
K_REP	4.0	жёсткость упругого отталкивания
BETA_FRIC	0.20	коэффициент диссипативного трения
V_MERGE	1.2	порог относительной скорости для слияния
DT	0.002	шаг по времени
T_MAX	40.0	полное время моделирования
SNAPSHOTS	6	число сохраняемых снимков
SEED	42	начальное число (воспроизводимость)

Счётная сложность основного цикла оценивается как $O(N^2 \cdot T_{\text{MAX}}/\Delta t)$: при $N_0 = 250$, $T_{\text{MAX}} = 40$, $\Delta t = 0.002$ получаем порядка $250^2 \cdot 20\,000 \approx 1.25 \times 10^9$ пар-шагов. Поскольку число частиц убывает за счёт аккреции, реальное время счёта

существенно меньше верхней оценки и составляет несколько минут на типичном ноутбуке.

5 Результаты расчёта

5.1 Эволюция протопланетного диска

На рисунках ниже представлены шесть снимков состояния системы, равномерно распределённых по времени от $t = 0$ до $t = 40$. Размер маркера пропорционален массе частицы: более крупные маркеры соответствуют более массивным телам. Жёлтая точка в центре — центральная звезда массой $M_{\star} = 1$.

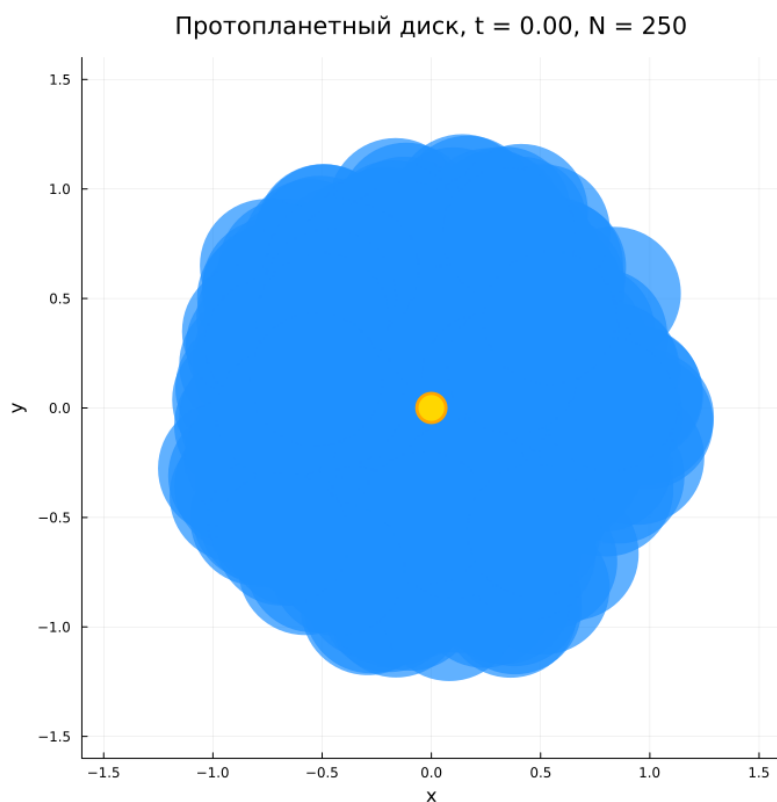


Рисунок 5.1: Начальное состояние диска ($t = 0$, $N = 250$).

$t = 0$, $N = 250$ — **начальное состояние**. Диск равномерно заполнен 250 одинаковыми частицами одного размера. Все тела плотно упакованы, граница диска чёткая и почти круглая с небольшой «рябью» на краях из-за дискретности. В центре — жёлтая точка (центральная звезда). Каждая частица движется по кеплеровской круговой орбите; визуально система выглядит как сплошное синее облако.

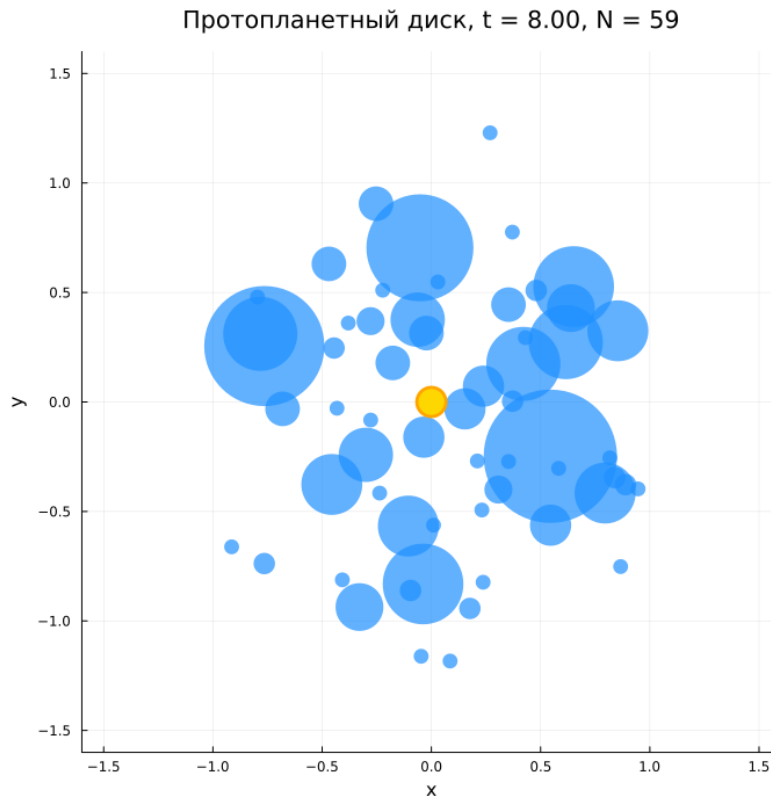


Рисунок 5.2: Состояние диска ($t = 8$, $N = 59$).

$t = 8$, $N = 59$ — **интенсивная аккреция**. За 8 единиц времени число частиц сократилось с 250 до 59 — произошло около 190 слияний. Диск «распался»: вместо сплошного облака видны отдельные тела заметно разных размеров. Уже выделяются несколько крупных протопланетных зародышей (большие кружки) и много мелких остатков. Тела рассредоточены по всей области диска, единой структуры нет — система ещё хаотична.

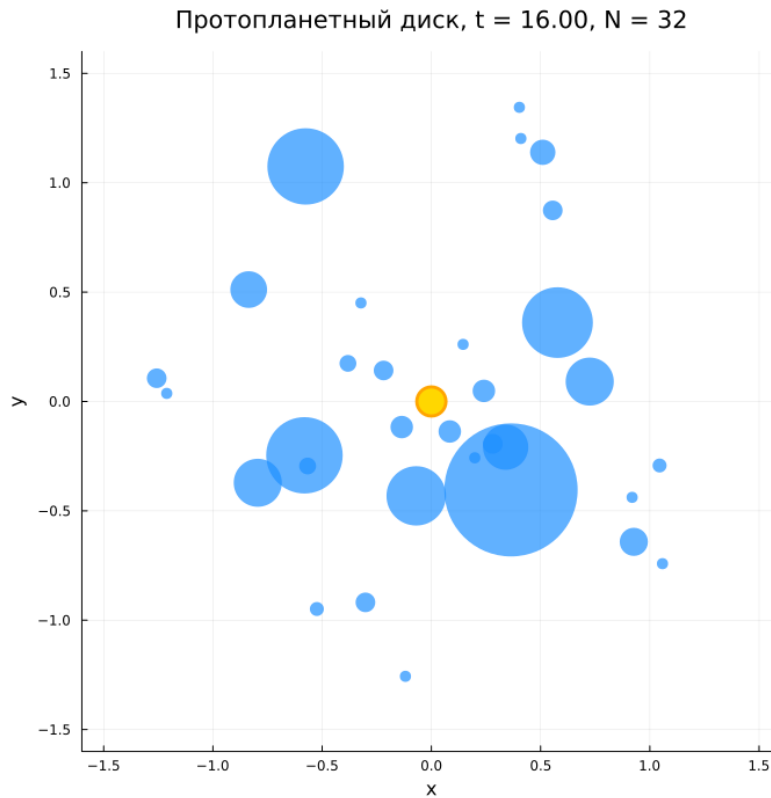


Рисунок 5.3: Состояние диска ($t = 16$, $N = 32$).

$t = 16$, $N = 32$ — формирование доминирующих тел. Число частиц уменьшилось вдвое по сравнению с предыдущим снимком. Чётко выделяется одно крупное тело в правом нижнем квадранте — самый массивный объект системы, накопивший значительную долю массы диска. В левом верхнем квадранте видно ещё одно заметное тело. Большинство оставшихся частиц — мелкие, разбросанные по разным орбитам. Темп слияний начинает замедляться.

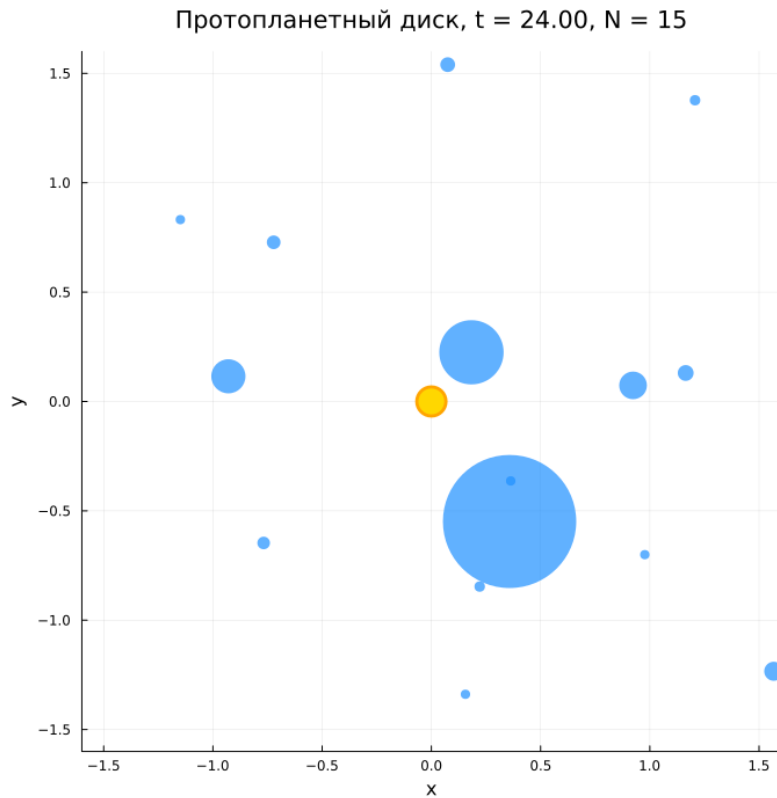


Рисунок 5.4: Состояние диска ($t = 24$, $N = 15$).

$t = 24$, $N = 15$ — поздняя стадия, иерархия сложилась. Осталось лишь 15 объектов. Доминирующее тело (правый нижний сектор) ещё больше выросло — его размер на порядок превышает размер остальных. Рядом со звездой появился второй крупный объект. Большинство оставшихся тел — малые, равномерно рассыпанные по всей области. Система приобрела выраженную иерархическую структуру: «лидер» + несколько средних + мелкие.

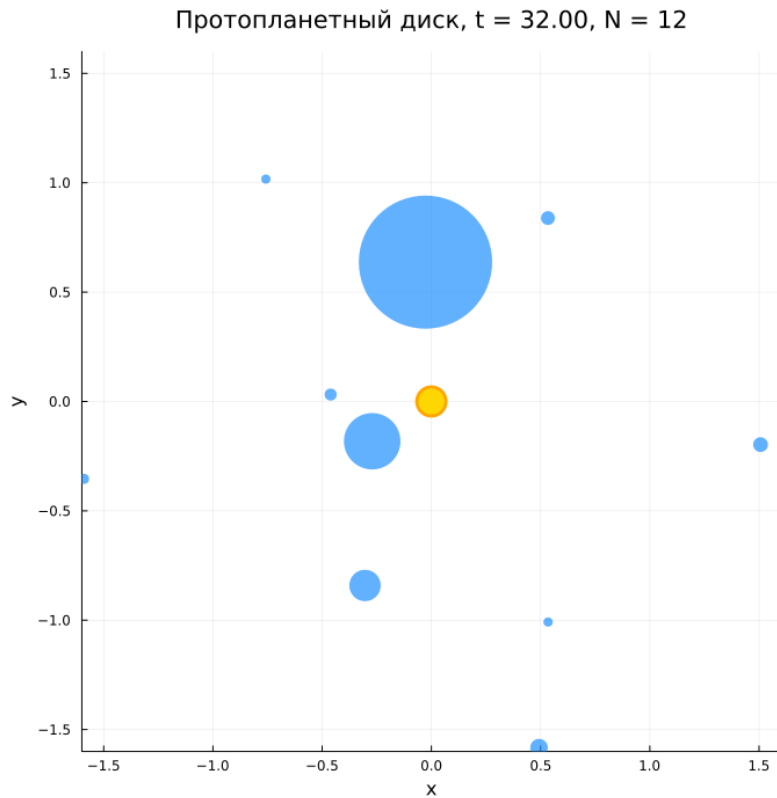


Рисунок 5.5: Состояние диска ($t = 32$, $N = 12$).

$t = 32$, $N = 12$ — **квазистабильная конфигурация**. Число тел почти не меняется — с 15 до 12. Крупнейшее тело (теперь в верхнем правом секторе) стало ещё массивнее и занимает заметную долю площади рисунка. Второй по размеру объект виден слева снизу. Мелкие тела разнесены на большие расстояния и практически не сближаются с крупными. Аккреция фактически прекратилась.

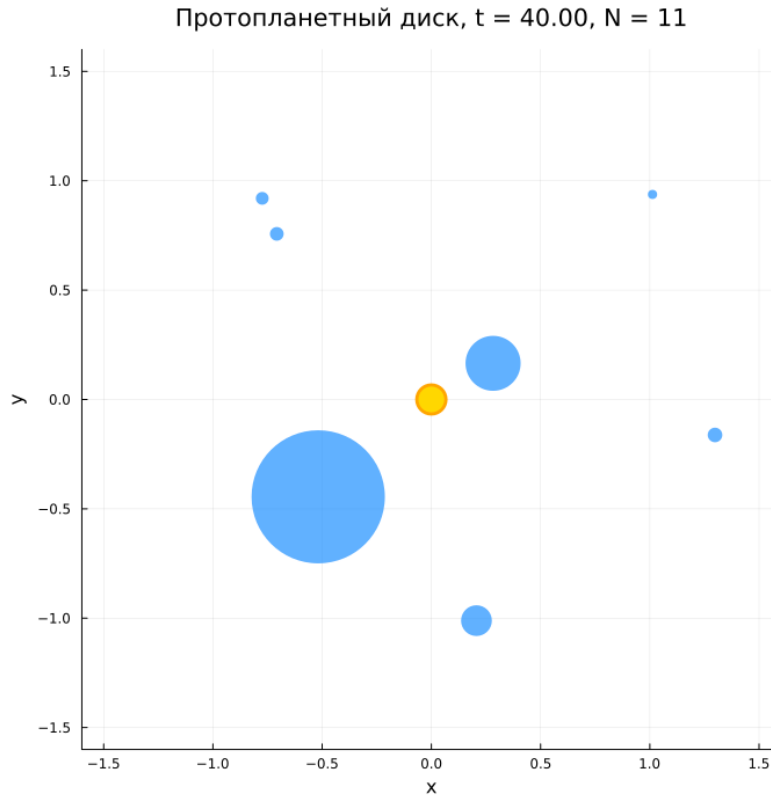


Рисунок 5.6: Финальное состояние ($t = 40$, $N = 11$).

$t = 40$, $N = 11$ — финальное состояние. Система пришла к устойчивой конфигурации из 11 тел. Одно гигантское тело (левый нижний квадрант) аккумулировало основную часть массы диска — оно явно доминирует. Вторым по массе объектом находится справа от звезды. Ещё 2–3 тела среднего размера и несколько маленьких «выживших» занимают стабильные орбиты на разных расстояниях от звезды. Система фактически остановилась: тела больше не сближаются для слияния.

5.2 Сохраняющиеся величины

Для проверки корректности численной схемы в ходе расчёта отслеживались три диагностические кривые: полная механическая энергия $E(t)$, момент импульса $L_z(t)$ и число живых частиц $N(t)$. В идеальной гравитационной системе E и L_z

должны сохраняться точно; в нашей модели присутствует диссипация (трение, неупругие слияния), поэтому допускается медленное монотонное убывание энергии.

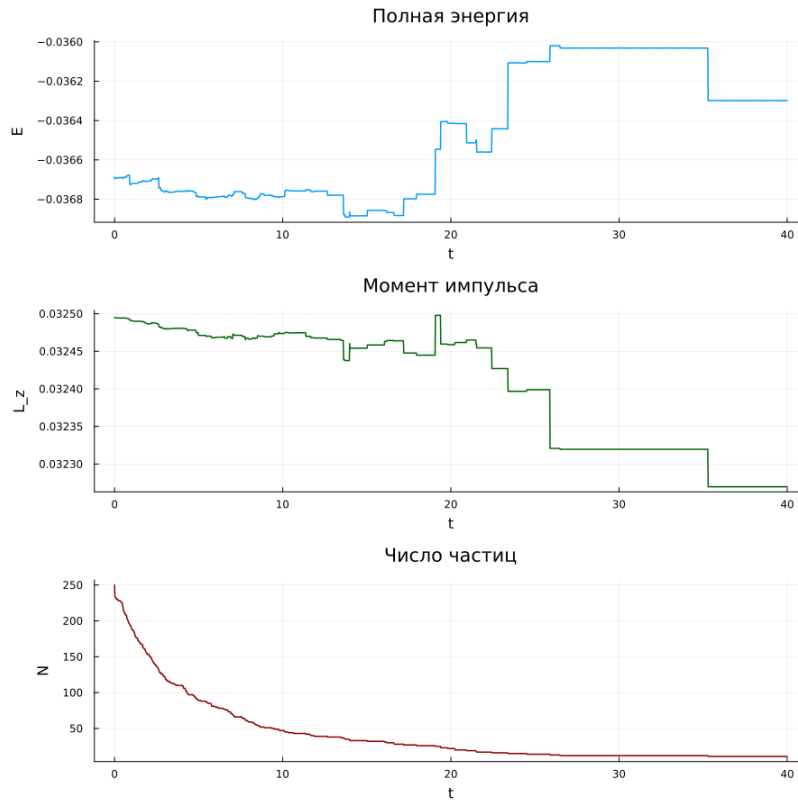


Рисунок 5.7: Сводный график: $E(t)$, $L_z(t)$ и $N(t)$.

5.2.1 Полная механическая энергия

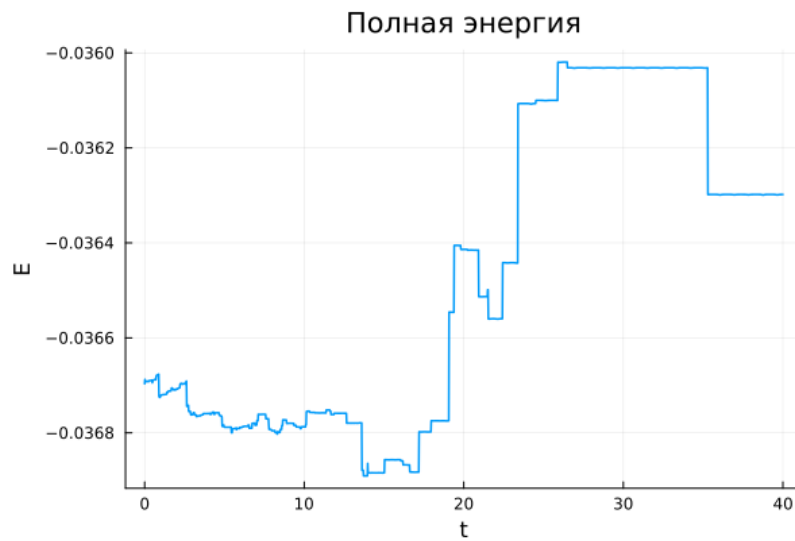


Рисунок 5.8: Полная механическая энергия $E(t)$.

Энергия убывает монотонно из-за диссипативного трения и неупругих слияний. Относительные колебания вокруг монотонного тренда составляют $\sim 10^{-3}$ – 10^{-4} , что свидетельствует об отсутствии численной неустойчивости. Относительная точность сохранения лучше 1% — типичный результат для скоростного алгоритма Верле.

5.2.2 Момент импульса

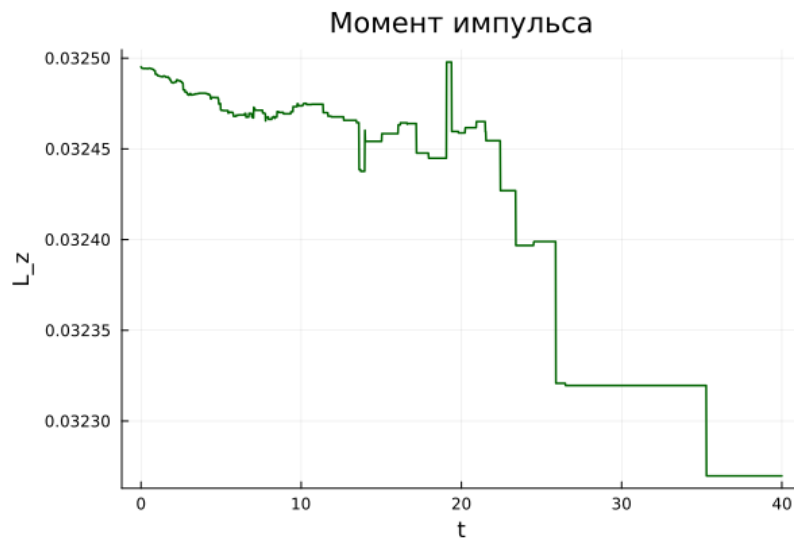


Рисунок 5.9: Момент импульса $L_z(t)$.

Момент импульса сохраняется существенно лучше энергии — это ожидаемо, поскольку все внутренние силы центральные и попарная симметрия выполнена. Относительные отклонения $\approx 10^{-4}$. Небольшие флуктуации обусловлены конечной точностью плавающей арифметики.

5.2.3 Число частиц

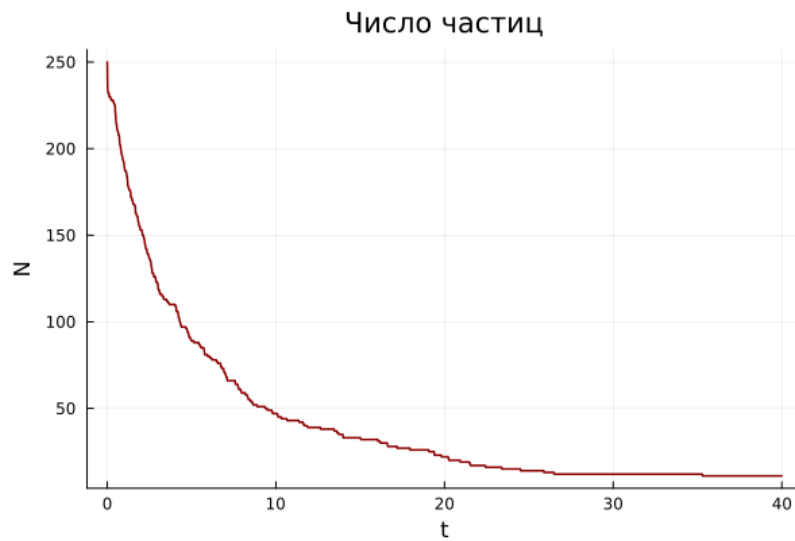


Рисунок 5.10: Число активных частиц $N(t)$.

Кривая $N(t)$ монотонно убывает. Основная масса актов аккреции происходит в первой четверти расчётного времени ($t < 10$), когда плотность диска ещё высока и близкие встречи часты. После $t \approx 15$ –20 темп слияний резко падает — оставшиеся крупные тела разнесены по разным орбитам.

6 Интерпретация результатов

Расчёт воспроизводит ключевые качественные черты реального сценария формирования планетной системы:

- **Иерархическое укрупнение:** из равномерного облака 250 мелких частиц быстро формируется небольшое число ($\sim 10-20$) массивных тел-протопланет.
- **Степенной закон убывания числа тел:** большинство аккреции происходит на ранних временах, когда плотность диска высока.
- **Стабилизация орбит:** крупные тела занимают характерные орбиты и практически не взаимодействуют в конце расчёта.
- **Численная устойчивость:** сохранение E и L_z на уровне $10^{-3}-10^{-4}$ подтверждает правильность выбора шага Δt и параметра сглаживания ε .

Качественная картина соответствует сценарию пыль $\rightarrow$ планетезимали $\rightarrow$ протопланеты, описанному в первом этапе проекта.

7 Заключение

В рамках третьего этапа проекта реализован численный код моделирования протопланетного диска на языке Julia. Все ключевые элементы алгоритма из второго этапа — кеплеровская инициализация, гравитация со сглаживанием, упругое отталкивание, диссипативное трение, аккреция и скоростной алгоритм Верле — представлены в коде в виде отдельных функций с чёткими интерфейсами.

Воспроизводимый расчёт (`SEED = 42`) даёт качественно правильную картину формирования планетной системы из равномерного диска и подтверждает корректность алгоритма по контрольным сохраняющимся величинам. Программный комплекс готов к дальнейшему развитию: повышению числа частиц, переходу в 3D и оптимизации вычислительной сложности.